

Documentación de propuesta técnica del software RentaMovil

Nicole Dayana Ramirez Vargas – Thiago Fabian Rojas Guevara – David Santiago Erez

Rodriguez

Servicio Nacional de Aprendizaje (SENA)

Programa de formación: Análisis y Desarrollo de Software

Asignatura o ficha: 3145556

Fecha de entrega: 9 de octubre de 2026 – Fecha de elaboración: 9 de octubre de 2026

Resumen

Este documento propone y justifica la solución técnica de RentaMovil antes de su diseño detallado: las alternativas evaluadas para cada decisión arquitectónica relevante, la pila tecnológica elegida, la arquitectura a alto nivel, la viabilidad del proyecto, la metodología de desarrollo y un cronograma propuesto por épica. No sustituye los registros de decisión arquitectónica (ADR) del repositorio de documentación, que son la fuente de verdad: los resume, los relaciona entre sí y los presenta como una propuesta única y coherente para evaluadores y equipo de desarrollo.

Palabras clave: RentaMovil, propuesta técnica, arquitectura de microservicios, arquitectura hexagonal, decisiones arquitectónicas, viabilidad, metodología Scrum.

1. Introducción

1.1 Propósito, audiencia y alcance

Este documento está dirigido a docentes, evaluadores y al equipo de desarrollo. Su propósito es consolidar, en un único documento, las decisiones técnicas ya registradas como ADR (Architecture Decision Record) en el repositorio *rtm-docs*, junto con la pila tecnológica, la viabilidad y la planificación propuestas para el proyecto. Cubre el backend de microservicios, la aplicación web de cliente y administración, y la aplicación móvil de cliente.

1.2 Objetivos

Objetivo general: proponer y justificar la solución técnica de RentaMovil, documentando las alternativas evaluadas, la arquitectura elegida y su viabilidad para el equipo y el tiempo disponible en el programa de formación.

Objetivos específicos:

- Registrar las alternativas consideradas para cada decisión arquitectónica relevante y la razón de la elección final.
- Justificar la pila tecnológica propuesta para backend, frontend web, aplicación móvil y base de datos.
- Evaluar la viabilidad técnica, operativa y económica del proyecto.
- Describir la metodología de desarrollo acordada por el equipo.
- Proponer un cronograma de alto nivel y los riesgos identificados con su mitigación.

1.3 Relación con los demás documentos del proyecto

Este documento se ubica entre la especificación de requisitos (documento 1) y el documento de análisis de vistas dinámicas (documento 2), que establecen *qué* debe hacer el sistema, y la

documentación de diseño del software (documento 4), que detalla *cómo* se construye (interfaces gráficas, modelo arquitectónico, patrones y modelo de datos). La propuesta técnica responde la pregunta intermedia: *qué solución se eligió y por qué*, antes de entrar al diseño detallado (RentaMovil, 2026, 01-context/overview.md; 03-product/vision.md).

2. Alternativas de solución evaluadas

Cada decisión arquitectónica relevante del proyecto quedó registrada como un ADR (Architecture Decision Record) en *05-architecture/decisions/records/*. Esta sección resume, para cada una, la alternativa elegida, las alternativas descartadas y la razón de la elección. Todas las decisiones aquí descritas tienen estado *Proposed* en el repositorio: son la arquitectura objetivo del proyecto, no necesariamente el estado actual de cada línea de código.

2.1 Descomposición del backend en microservicios

Alternativa	Estado	Razón
Un servicio por Bounded Context (7 servicios)	Elegida	Alinea el ownership del código con el lenguaje de dominio y los eventos ya definidos en el modelo de dominio
Un servicio por feature de frontend	Descartada	Acopla los límites del backend a la interfaz de usuario y duplica el ownership de dominio
Monolito grande indefinido	Descartada	No establece un ownership de dominio independiente por equipo/servicio

La contrapartida de la decisión elegida es que las migraciones sobre la base de datos SQL Server compartida requieren coordinación explícita entre servicios, y el despliegue independiente de cada servicio sigue siendo una capacidad futura, no actual (RentaMovil, 2026, ADR-002-microservices-decomposition.md).

2.2 Arquitectura del cliente móvil

Alternativa	Estado	Razón
Feature-Based + Component-Driven + Expo Router	Elegida	Ownership claro por funcionalidad de negocio y reutilización de componentes de interfaz
Carpetas organizadas solo por capas	Descartada	Dispersa el comportamiento de cada funcionalidad por todo el proyecto
Lógica de negocio dentro de los archivos de ruta	Descartada	Acopla la navegación al comportamiento de negocio

Como consecuencia, las nuevas funcionalidades del cliente móvil se ubican bajo *src/features/*, la interfaz compartida bajo *src/shared/*, y *src/app/* queda reservado solo para navegación (RentaMovil, 2026, ADR-003-mobile-architecture.md).

2.3 Gestión de estado en el cliente

Alternativa	Estado	Razón
Estado local + Context por funcionalidad + Zustand para estado transversal	Elegida	Ownership explícito del estado y adopción gradual, sin forzar un único mecanismo global
Zustand para todo el estado del cliente	Descartada	Fomenta acoplamiento global innecesario
Context para todo el estado del cliente	Descartada	Puede causar renderizados amplios y ownership poco claro

Los datos propiedad del servidor permanecen en los servicios backend y no se tratan como fuente de verdad en el cliente; esta decisión no autoriza acceso directo del cliente a la persistencia (RentaMovil, 2026, ADR-004-state-management.md).

2.4 Autenticación y autorización en la web

Alternativa	Estado	Razón
Autorización en el backend + route guards en el frontend	Elegida	Segura y usable; la decisión final de acceso siempre es del backend
Autorización resuelta solo en el frontend	Descartada	No protege los datos expuestos por la API
Supabase Auth como emisor de identidad	Descartada	Entra en conflicto con la política de seguridad del proyecto

El frontend web se integra con el servicio de identidad a través de la API backend, almacena los refresh tokens en cookies *httpOnly*, y usa guards de ruta para la navegación; los tokens de acceso se emiten como JWT firmados con RS256 (RentaMovil, 2026, ADR-005-web-authentication-and-authorization.md).

2.5 Arquitectura del frontend web

Alternativa	Estado	Razón
Estructura de funcionalidad estandarizada (Feature + Component + Route-Based)	Elegida	Predecible y escalable a medida que crece el número de pantallas
Preservar todas las convenciones de carpetas existentes	Descartada para código nuevo	Mantiene el riesgo de imports inconsistentes en sistemas sensibles a mayúsculas/minúsculas
Estructura de proyecto organizada solo por capas	Descartada	Debilita el ownership de cada funcionalidad

El frontend web mezclaba previamente convenciones de carpetas distintas (*Pages/pages*, *Services/services*); el código nuevo adopta nombres en minúsculas y *services/* en plural, migrando el resto de forma incremental (RentaMovil, 2026, ADR-006-web-frontend-architecture.md).

2.6 Frontera entre frontend y backend

Alternativa	Estado	Razón
Los clientes consumen únicamente las REST API del backend propio	Elegida	Centraliza las reglas de negocio y la autorización en el backend
Acceso directo del frontend a Supabase u otro proveedor de datos	Descartada como arquitectura objetivo	Evade el ownership de dominio y la política de seguridad del backend
Acceso directo del frontend a SQL Server	Descartada	Inseguro e imposible de gobernar

Cualquier uso temporal de Supabase en los clientes actuales se trata como deuda técnica o punto de integración temporal, y no debe expandirse sin actualizar el ADR correspondiente (RentaMovil, 2026, ADR-007-frontend-backend-boundary.md).

2.7 Alcance de rutas del MVP

Alternativa	Estado	Razón
Ocultar o remover del frontend las rutas de contratos de alquiler	Elegida	Alinea la navegación activa con el alcance real del MVP
Mantener activas las rutas de contrato existentes	Descartada	Tergiversa el alcance del MVP y crea flujos no soportados por el backend
Agregar los contratos al MVP de manera informal	Descartada	Rompe la gobernanza de alcance y la trazabilidad con los requisitos

El catálogo de rutas distingue rutas públicas, de cliente, administrativas y legacy; agregar los contratos de alquiler al alcance requeriría un cambio de alcance aprobado y actualizar producto, requisitos, dominio, API y arquitectura (RentaMovil, 2026, ADR-008-mvp-route-boundary.md).

2.8 Esquema y migraciones de base de datos

Alternativa	Estado	Razón
Un schema por servicio + un repositorio de migraciones por servicio	Elegida	El ownership de cada tabla queda explícito en su nombre; los repos nunca colisionan entre sí
Todas las tablas en <i>dbo</i> , un repositorio de migración por servicio	Descartada	El ownership queda solo por convención; cualquier servicio puede escribir en cualquier tabla
Un repositorio de migración único y compartido entre todos los servicios	Descartada	Acopla el release de todos los equipos a un solo repositorio
Una base de datos independiente por servicio	Descartada	Contradice la base de datos compartida y pierde las llaves foráneas entre contextos

La base de datos sigue siendo una única instancia de SQL Server compartida (*rentamovil*), con un schema por servicio de negocio (*iam*, *fleet*, *telemetry*, *booking*, *payment*) y un repositorio *rtm-<servicio>-db* por servicio. El orden de ejecución entre repositorios es fijo (*iam* → *fleet-maintenance* → *telemetry-gps* → *booking-reservation* → *payment-billing*) para que las llaves foráneas entre servicios siempre apunten a tablas ya existentes (RentaMóvil, 2026, ADR-009-schema-per-service-and-migration-repos.md).

3. Pila tecnológica propuesta

Capa	Tecnología	Justificación
Backend	Java + Spring Boot, Arquitectura Hexagonal	Separa las reglas de negocio de Spring, SQL Server y el transporte; es el estándar del programa de formación
Base de datos	Microsoft SQL Server (instancia compartida, un schema por servicio)	Única tecnología de persistencia declarada para el proyecto; migrada con Liquibase por servicio
Web (cliente y administración)	React + Vite	Permite una arquitectura por funcionalidad y componentes

Capa	Tecnología	Justificación
		reutilizables, con tiempos de compilación rápidos
Móvil (cliente)	Expo + React Native	Un solo código fuente para Android/iOS, con Expo Router para la navegación
Autenticación	JWT firmado con RS256, emitido por el servicio de identidad	Permite que cada microservicio verifique el token de forma independiente, sin consultar al emisor en cada petición
Comunicación	REST síncrono entre cliente y backend; eventos de dominio asíncronos entre servicios, con entrega <i>at-least-once</i>	Exige que los flujos de pago, alquiler, telemetría y notificación sean idempotentes

(RentaMovil, 2026, 05-architecture/overview.md)

4. Arquitectura propuesta a alto nivel

La arquitectura es cliente-servidor: las aplicaciones web y móvil de cliente, y la interfaz web administrativa, consumen las API del backend mediante contratos publicados (OpenAPI). El backend se organiza en siete Bounded Contexts, agrupados en servicios desplegables:

Bounded Context	Servicio objetivo
Identity & Access	identity-access-service
Fleet & Maintenance	fleet-maintenance-service
Booking & Reservation	booking-reservation-service
Rental Execution	rental-execution-service (ejecuta dentro de booking-reservation)
Payment & Billing	payment-billing-service
Telemetry & GPS	telemetry-gps-service
Notification	notification-service (ejecuta dentro de booking-reservation)

audit es infraestructura transversal escrita directamente por los siete contextos, no un Bounded Context propio con pantalla o microservicio dedicado.

Principios arquitectónicos: contratos de API antes de implementar endpoints públicos (*API first*); aislamiento de dominio respecto a Spring, SQL Server y el transporte; ownership delimitado por contexto; dependencias controladas únicamente vía REST o eventos de dominio publicados, nunca acceso interno directo; seguridad incorporada desde el diseño; evolución incremental del sistema.

Patrones arquitectónicos: Arquitectura Hexagonal (adoptada en el backend), Feature-Based y Component-Driven (adoptados en el cliente móvil), API Gateway (frontera objetivo del sistema), Saga/Outbox y Circuit Breaker (a evaluar para flujos entre contextos y llamadas entre servicios desplegados de forma independiente, respectivamente). Una eventual migración de la base de datos compartida a una base de datos por servicio requeriría un nuevo ADR (RentaMovil, 2026, 05-architecture/overview.md).

5. Viabilidad

5.1 Viabilidad técnica

El equipo cuenta con experiencia previa en Java/Spring Boot, React y bases de datos relacionales, reforzada durante el programa de formación. El stack elegido (sección 3) usa herramientas de uso extendido en la industria y con documentación abundante, lo que reduce el riesgo de adopción. La principal complejidad técnica identificada es la coordinación de migraciones entre los repositorios *rtm-<servicio>-db*, mitigada mediante el orden de ejecución fijo descrito en la sección 2.8.

5.2 Viabilidad operativa

El proyecto se desarrolla con un equipo de 3 integrantes bajo la metodología descrita en la sección 6, con herramientas gratuitas o con nivel gratuito disponible (Jira, GitHub, Docker). No se identifican dependencias de infraestructura de producción fuera del alcance de un entorno de desarrollo y pruebas local o en contenedores (RentaMovil, 2026, 00-governance/agile- conventions.md).

5.3 Viabilidad económica

Al tratarse de un proyecto académico, no existe presupuesto asignado para infraestructura de producción ni para licencias comerciales. El stack tecnológico propuesto (sección 3) usa en su totalidad herramientas de código abierto o con nivel gratuito suficiente para el alcance del MVP (Java, Spring Boot, SQL Server Developer Edition, React, Expo, Docker). No se realizó un análisis de costo-beneficio formal de despliegue en producción, por estar fuera del alcance académico del programa de formación.

6. Metodología de desarrollo

El equipo acordó trabajar bajo **Scrum**, con sprints de **una semana** (lunes a viernes) para un equipo de **3 personas**. Las ceremonias se comprimieron para ajustarse a la duración corta del sprint:

Ceremonia	Momento	Duración máxima	Participantes
Sprint Planning	Lunes	1 hora	Equipo completo
Daily Stand-up	Diario	15 minutos	Equipo completo
Backlog Refinement	Miércoles	30 minutos	Equipo completo
Sprint Review	Viernes	30 minutos	Equipo + Product Owner
Sprint Retrospective	Viernes (tras la Review)	30 minutos	Equipo completo

Cada retrospectiva debe producir al menos una acción de mejora con responsable y fecha límite. La estimación de historias usa la escala **Fibonacci** (1, 2, 3, 5, 8, 13) mediante **Planning Poker**; una historia estimada en 8 o 13 puntos debe dividirse antes de entrar al sprint. El backlog se gestiona en **Jira**, con tablero de columnas Backlog → Ready → In Progress → In Review → Done (RentaMovil, 2026, 00-governance/agile-conventions.md).

7. Recursos necesarios

7.1 Recursos humanos

El equipo está compuesto por 3 integrantes, con los roles ya definidos en la especificación de requisitos del proyecto:

Integrante	Rol	Responsabilidad
Nicole Dayana Ramírez Vargas	Líder	Liderar, planificar y supervisar la entrega del proyecto
David Santiago Erez Rodríguez	Desarrollador	Programar el aplicativo web
Thiago Fabian Rojas Guevara	Analista	Análisis de información, diseño y programación

(RentaMovil, 2026, Documento 1 — Especificación de Requisitos de Software, sección 1.7)

7.2 Recursos de software

Categoría	Herramientas
Backend	Java (JDK), Spring Boot, Maven/Gradle
Base de datos	Microsoft SQL Server Developer Edition, Liquibase
Frontend web	Node.js, React, Vite
Aplicación móvil	Expo, React Native
Control de versiones	Git, GitHub
Gestión del backlog	Jira

Categoría	Herramientas
Contenedores	Docker
Entornos de desarrollo	IDE del equipo (IntelliJ IDEA, Visual Studio Code)

Todas las herramientas listadas son de código abierto o cuentan con un nivel gratuito suficiente para el alcance del MVP, consistente con lo descrito en la viabilidad económica (sección 5.3).

7.3 Recursos de hardware

El proyecto no requiere servidores de producción: el desarrollo, las pruebas y la integración se realizan en los equipos de cómputo de los integrantes del equipo y en contenedores Docker locales. Las pruebas de la aplicación móvil se realizan sobre emuladores (Android/iOS) o dispositivos físicos propios del equipo, sin infraestructura adicional. La excepción es el módulo de Telemetría y GPS: para validar la integración real con un dispositivo GPS, el equipo adquirió un módulo físico y sus componentes, descritos en el presupuesto de la sección 7.4.

7.4 Materiales y gastos únicos

Los siguientes son los gastos en los que el equipo ha incurrido hasta la fecha, con cargo a recursos propios de sus integrantes, principalmente para la validación física del módulo de Telemetría y GPS:

Ítem	Descripción	Costo aproximado (COP)
Módulo y materiales GPS	Dispositivo GPS y componentes electrónicos para el prototipo de rastreo	\$300.000
Insumos electrónicos adicionales	Cables, conectores, protoboard y carcasa	\$50.000
Transporte	Desplazamiento para la compra de materiales	\$50.000

Ítem	Descripción	Costo aproximado (COP)
Imprevistos y varios	Gastos adicionales no planeados durante la compra e integración	\$100.000
Subtotal gastos únicos		\$500.000

7.5 Costos recurrentes y costo total estimado del proyecto

Tomando como duración total del proyecto **18 meses** (el tiempo completo del equipo en el programa de formación, no solo las semanas de ejecución de sprints de la sección 8), se estiman además los siguientes costos recurrentes mensuales: el salario de cada integrante del equipo y los servicios de internet y energía eléctrica usados durante el desarrollo.

Concepto	Costo mensual (COP)	Meses	Costo total (COP)
Salario del equipo (3 integrantes × \$800.000)	\$2.400.000	18	\$43.200.000
Servicios (internet y energía, equipo completo)	\$150.000	18	\$2.700.000
Subtotal costos recurrentes			\$45.900.000

Resumen	Costo (COP)
Gastos únicos (sección 7.4)	\$500.000
Costos recurrentes (18 meses)	\$45.900.000
Costo total estimado del proyecto	\$46.400.000

Estas cifras son un estimado de carácter académico, con fines ilustrativos del costo real que representaría el tiempo del equipo durante el programa de formación; no corresponden a un presupuesto de despliegue en producción, que queda fuera del alcance académico del programa, como ya se indica en la viabilidad económica (sección 5.3).

8. Estado actual y cronograma propuesto

El backlog del proyecto, documentado en *04-requirements/user-stories.md*, está organizado en 7 épicas (una por Bounded Context) con 31 historias de usuario en total, estimadas entre 2 y 5 puntos cada una.

Esta propuesta sugiere ejecutar las épicas siguiendo el mismo orden de dependencia que ya rige las migraciones de base de datos entre servicios (sección 2.8), de modo que ningún servicio dependa de tablas o contratos que todavía no existen:

Orden	Épica (Bounded Context)	Dependencia	Duración estimada
1	Identity & Access	Ninguna — autenticación requerida por el resto	2 sprints
2	Fleet & Maintenance	Identity & Access (roles y permisos)	2 sprints
3	Telemetry & GPS	Fleet & Maintenance (vehículos a rastrear)	1 sprint
4	Booking & Reservation	Identity & Access, Fleet & Maintenance	2 sprints
5	Rental Execution	Booking & Reservation (reserva confirmada)	1 sprint
6	Payment & Billing	Booking & Reservation	2 sprints
7	Notification	Transversal a los eventos de las épicas anteriores	1 sprint
Total estimado			11 sprints (≈ 11 semanas)

Con sprints de una semana y un equipo de 3 personas, el proyecto completo se estima en alrededor de 11 semanas de ejecución, desde la primera Sprint Planning hasta el cierre de la última épica. La duración por épica es una estimación de planificación, proporcional a su

complejidad relativa; el equipo aún no registra datos de velocidad real (RentaMovil, 2026, 00-governance/agile-conventions.md), por lo que no se fijan fechas de calendario concretas — el inicio y cierre de cada sprint quedan sujetos a la Sprint Planning semanal, conforme a la metodología descrita en la sección 6.

9. Riesgos y mitigación

Riesgo	Origen	Mitigación propuesta
Migraciones de esquema descoordinadas entre servicios que comparten la base de datos	ADR-002, ADR-009	Orden de ejecución fijo entre repositorios de migración y revisión obligatoria antes de desplegar cambios de esquema
Referencias residuales a Supabase en el frontend	ADR-007	Tratarlas como deuda técnica explícita; no ampliarlas sin actualizar el ADR correspondiente
Rutas de contratos de alquiler expuestas fuera del alcance del MVP	ADR-008	Ocultar o remover la navegación activa hacia esas rutas hasta que exista un cambio de alcance aprobado
Convenciones de carpetas inconsistentes en el frontend web (mayúsculas/minúsculas)	ADR-006	Exigir la convención estandarizada en código nuevo; migración incremental del código existente
Backlog con cronograma de sprints sin asignar	04-requirements/user-stories.md	Priorizar la asignación de sprints siguiendo el orden de dependencia propuesto en la sección 7
Desacuerdo de estimación entre integrantes del equipo	00-governance/agile-conventions.md	Planning Poker con regla de discusión obligatoria ante diferencias de 2 o más niveles en la escala Fibonacci

10. Conclusiones

La solución técnica propuesta para RentaMovil está respaldada por nueve decisiones arquitectónicas ya registradas y evaluadas contra alternativas concretas, con una pila

tecnológica de uso extendido en la industria y viable para un equipo de tres integrantes dentro del programa de formación. Las 31 historias de usuario del backlog ya están refinadas; la ejecución de sprints sigue el orden de dependencia entre épicas propuesto en este documento.

Referencias

RentaMovil. (2026). 01-context/overview.md y 03-product/vision.md [Visión y contexto del proyecto]. Repositorio de documentación RentaMovil.

RentaMovil. (2026). ADR-002-microservices-decomposition.md [Registro de decisión arquitectónica]. Repositorio de documentación RentaMovil.

RentaMovil. (2026). ADR-003-mobile-architecture.md [Registro de decisión arquitectónica]. Repositorio de documentación RentaMovil.

RentaMovil. (2026). ADR-004-state-management.md [Registro de decisión arquitectónica]. Repositorio de documentación RentaMovil.

RentaMovil. (2026). ADR-005-web-authentication-and-authorization.md [Registro de decisión arquitectónica]. Repositorio de documentación RentaMovil.

RentaMovil. (2026). ADR-006-web-frontend-architecture.md [Registro de decisión arquitectónica]. Repositorio de documentación RentaMovil.

RentaMovil. (2026). ADR-007-frontend-backend-boundary.md [Registro de decisión arquitectónica]. Repositorio de documentación RentaMovil.

RentaMovil. (2026). ADR-008-mvp-route-boundary.md [Registro de decisión arquitectónica]. Repositorio de documentación RentaMovil.

RentaMovil. (2026). ADR-009-schema-per-service-and-migration-repos.md [Registro de decisión arquitectónica]. Repositorio de documentación RentaMovil.

RentaMovil. (2026). 05-architecture/overview.md [Visión general de arquitectura]. Repositorio de documentación RentaMovil.

RentaMovil. (2026). 00-governance/agile-conventions.md [Convenciones ágiles del equipo].

Repositorio de documentación RentaMovil.

RentaMovil. (2026). 04-requirements/user-stories.md [Historias de usuario y estado del

backlog]. Repositorio de documentación RentaMovil.